DISTRIBUTED MULTI-MODEL ANALYTICS FOR E-COMMERCE DATA

PRACTICAL STUDY, REVISION & VIVA GUIDE

Kinyarwanda explanations • Simple English code comments
Windows PowerShell • MongoDB • HBase • Apache Spark

November 2026 | Prepared from verified project outputs shared in the working session

A self-contained learning companion, not a substitute for your actual scripts or lecturer rubric.

How to use this guide

Iyi nyandiko yagenewe kugufasha gusobanukirwa umushinga kuva ku ntangiriro kugeza ku musozo, ndetse no kwitegura kubazwa (viva). Banza usome concept; reba code snippet; shyira mu magambo yawe icyo script ikora; hanyuma ugerageze commands ku mudasobwa yawe.

Evidence levels: (1) CONFIRMED = values and outcomes reported from your actual terminal; (2) ILLUSTRATIVE = examples of code or SQL to explain a technique, not claims that the specific snippet was executed; (3) CHECK BEFORE SUBMISSION = points that require final verification.

Project root on your PC: C:\Users\user\Desktop\FinalProjectBD. Run commands from this directory unless stated otherwise.

Quick navigation

• 1. Project purpose and architecture

• 2. Dataset generation and records

• 3. MongoDB data model and queries

• 4. HBase data modeling and access

• 5. Apache Spark setup and batch cleaning

• 6. Spark SQL and business results

• 7. Cohort analysis and careful interpretation

• 8. Integrated analytics: MongoDB + HBase + Spark

• 9. Visualizations and findings

• 10. Implementation problems and troubleshooting

• 11. Reproducibility and submission checklist

• 12. Explain the project in a viva

• 13. Practice quiz and hands-on revision

• 14. Glossary and final factual summary

Icyitonderwa / Important: The guide is intentionally longer than the lecturer's 6–8-page technical report. The technical report and this study guide have different purposes.

1. Project purpose and architecture

Mu Kinyarwanda: Umushinga ugamije kwerekana uko twatangira dukora amakuru y'ubucuruzi bwo kuri internet, tukayabika muri databases zitandukanye, tukayasesengura, hanyuma tugatanga ibisubizo bifasha gufata ibyemezo. MongoDB ibika documents; HBase ikita ku makuru y'amasomo ya browsing; Spark ikora cleaning, SQL no guhuza datasets. Ujye wibuka ko guhuza sources ari igikorwa cyihariye, si ukubona ko buri technology yakoze ukwawo gusa.

1.1 What problem are we solving?

Business question: Which products attract customer browsing interest, and how does this relate to their transaction performance? Mu Kinyarwanda: Ni ibihe bicuruzwa bisurwa cyane, kandi ayo makuru ahuriye he n'umubare w'ibicuruzwa biri muri transactions?

Objectives: generate synthetic e-commerce data; store document-oriented data in MongoDB; model session and product-date information in HBase; analyze and clean transactions in Spark; join data across sources; visualize and explain business findings.

1.2 Architecture in plain language

Layer	Technology / file	Purpose
Data generation	dataset_generator.py and JSON files	Create users, products, transactions and sessions
Document storage	MongoDB / ecommerce_db	Store entire synthetic dataset; aggregate product transactions
Column-family storage	HBase / fp_* tables	Store representative browsing-session subset and daily metrics
Batch processing	PySpark	Read NDJSON, clean data, use Spark SQL and cohort queries
Cross-source analytics	CSV exports + Spark JOIN	Join by product_id; compare browsing and transaction activity
Delivery	PNG charts, report, PPTX, study guide	Communicate methodology, results and caveats

Core data movement: Generator → MongoDB; Generator → HBase (5,000 selected sessions); Generator → Spark cleaned Parquet. MongoDB product and transaction summaries + HBase product metrics → CSV exports → Spark left joins → integrated product CSV → charts and report.

1.3 Why more than one database?

MongoDB is a document database: it naturally stores nested structures such as a transaction containing an items array. HBase is a wide-column database: it is useful for row-key-based access to browsing sessions and product metrics. Spark is a distributed processing engine, not the main permanent document store. The exercise demonstrates combining complementary systems rather than claiming any tool is always faster.

1.4 Actual project folders

```
FinalProjectBD/
  dataset_generator.py
  users.json, products.json, transactions.json, sessions_0.json ...
  hbase/        schema.hbase, load_hbase.py, queries.hbase
  spark/        prepare_transactions.py, spark_batch.py,
                spark_clean.py, spark_sql_queries.py, spark_cohort.py
  integration/  convert_hbase_to_csv.py, export_mongodb.py,
                spark_integrated_analysis.py, *.csv
  visualizations/charts/  chart1_*.png ... chart4_*.png
  report/       architecture.png, Technical_Report.docx,
                Technical_Report.pdf, build_presentation.py, *.pptx
```

Icyitonderwa / Important: Actual subfolder contents can differ if files were moved. Validate with Get-ChildItem; avoid submitting outputs containing private credentials.

2. Dataset generation and records

Mu Kinyarwanda: Synthetic data ni amakuru yakozwe na program, ntabwo ari amakuru yakusanyijwe ku bakiriya nyabo. Generator yakoze abantu 10,000, products 5,000, transactions 500,000 na sessions miliyoni 2. Icyo ugomba gusobanura muri viva ni itandukaniro rya transaction na session: transaction ni igikorwa cy'ubuguzi cyanditswe, naho session ni urugendo rw'umukiriya ku rubuga.

2.1 Confirmed dataset sizes

Entity	Count	What one record represents
users	10,000	One registered customer
products	5,000	One catalog product
categories	25	One product category
transactions	500,000	One transaction with nested items
sessions	2,000,000	One browsing session with page views and cart contents

The 2 million sessions were split into sessions_0.json through sessions_19.json (100,000 per file). All five collections were verified in MongoDB: total 2,515,025 documents.

2.2 How the generator models behavior

The original Python generator used session flows including home, search, category_listing, product_detail, cart, checkout and confirmation. Device types included mobile, desktop and tablet. Referrers included direct, email, social, search_engine and affiliate. The stated generation settings included 30% chance of adding to cart at an opportunity and 70% checkout completion probability. Discounts were applied in some transactions (20% chance; rates of 5%, 10%, 15% or 20%).

These are generator settings, not directly observed conversion or retention results. The event probabilities cannot simply be treated as the resulting session-level conversion rate, because paths can contain multiple steps and the sample was generated under specific constraints.

2.3 Understand the main fields

Record	Example fields	Why they matter
User	user_id, registration_date, geo_data.country	Join to transactions; build registration cohorts
Product	product_id, name, category_id, base_price	Join product catalog to browsing and quantities
Transaction	transaction_id, user_id, status, items, total	Calculate totals, quantities and status breakdown
Session	session_id, user_id, start_time, page_views, cart_contents	Compute product-detail views and cart metrics

```
# Example: one transaction has multiple items.
transaction = {
  "transaction_id": "txn_example",
  "user_id": "user_000001",
  "status": "completed",
  "items": [
    {"product_id": "prod_00001", "quantity": 2, "subtotal": "120.00"}
  ],
  "total": "108.00"
}
```

The example above is illustrative, not a copied generator record. The transaction-level total may be lower than the sum of item subtotals because of discount.

2.4 Why validation matters

JSON files need syntactic validation, expected record counts and key checks before import. Streaming JSON readers such as ijson avoid loading an entire large JSON array into memory. For each file, check required keys, schema compatibility and representative sample records.

```
# Run from FinalProjectBD; verify the package is installed.
python -c "import ijson; print('ijson available')"

# Show one user record without loading all users into RAM.
python -c "import ijson,pprint; f=open('users.json','rb'); pprint.pp(next(ijson.items(f,'item'))); f.close()"
```

3. MongoDB data model and queries

Mu Kinyarwanda: MongoDB yabikaga amakuru mu buryo bwa JSON-like documents. Transaction imwe ishobora kugira items nyinshi; ni yo mpamvu $unwind iba ingenzi mu kubara ibicuruzwa ku giti cyabyo. Ntukitiranye agaciro ka item mbere ya discount n'amafaranga ya nyuma ya transaction, cyangwa ngo wite ayo mafaranga profit.

3.1 Database and verified collection sizes

Database name: ecommerce_db. MongoDB Community on localhost:27017. Verified collections: categories 25; products 5,000; sessions 2,000,000; transactions 500,000; users 10,000.

Document structure lets transactions store an embedded items array. A product record includes product_id, name, category_id, base_price, current_stock, is_active and price_history. Monetary fields were seen as MongoDB Decimal128; preserve decimals when calculating values.

```
# Read collection names and approximate document counts.
python -c "from pymongo import MongoClient; db=MongoClient('mongodb://localhost:27017/')['ecommerce_db'];
print([(n,db[n].estimated_document_count()) for n in sorted(db.list_collection_names())])"
```

3.2 Two MongoDB aggregation patterns (illustrative)

Aggregation pipelines run computations inside MongoDB. The following are educational examples aligned with the model. They are NOT a verbatim copy of the project's earlier MongoDB scripts, which are not present in this guide.

```
// Example A: count transactions by status.
db.transactions.aggregate([
  { $group: {
      _id: "$status",
      transactions: { $sum: 1 },
      transactionValue: { $sum: "$total" }
  }},
  { $sort: { transactions: -1 } }
]);
```

Stage $group groups by status and computes sums; $sort ranks the groups. If total is stored as Decimal128, the aggregate value stays precise.

```
// Example B: quantities and item subtotals per product.
db.transactions.aggregate([
```

```
  { $unwind: "$items" },
  { $group: {
    _id: "$items.product_id",
    quantity: { $sum: "$items.quantity" },
    itemValueBeforeDiscount: { $sum: "$items.subtotal" }
  }},
  { $sort: { quantity: -1 } },
  { $limit: 10 }
]);
```

$unwind makes each element of items act like a separate record; then $group aggregates by product ID. This is different from summing transaction totals, because transaction totals include discounts and can contain several items.

3.3 Important distinction: quantity is not revenue

In export_mongodb.py, completed_quantity counts quantities in items of transactions whose status equals completed. completed_item_value sums item subtotals BEFORE transaction-level discounts. Therefore it is not proven net sales revenue. Shipped/delivered statuses were kept separate in the Spark status summary.

3.4 Project evidence for MongoDB

Verified exports: mongodb_products.csv has 5,000 catalog products; mongodb_product_sales.csv has 4,766 product IDs found inside transaction items. Thus 234 catalog products did not appear inside the processed transaction items. The export script read 500,000 transactions.

4. HBase data modeling and access

Mu Kinyarwanda: HBase ikoresha row key kugira ngo ishakishe amakuru. Kuri sessions twakoresheje user ID n'igihe cyahinduwe; kuri product metrics twakoresheje product ID n'itariki. Muri sessions 5,000 twakuyemo views na quantity y'ibicuruzwa byari muri cart. Cart quantity ntabwo ari ibicuruzwa byamaze kugurwa.

4.1 Why HBase and what is a column family?

HBase stores cells identified by row key, column family, qualifier and timestamp. A column family groups related fields. In this project, session info, page views and cart contents were separated logically for a row-key-oriented representation.

Table	Column families	Confirmed purpose
fp_user_sessions	info, pv, cart	Session metadata, page views and cart contents
fp_product_metrics	m	Per-product/per-date views and cart_quantity

4.2 Row-key design

```
# Session row key (from loader logic):
user_id + "#" + reverse_time + "#" + session_id

# Product metrics row key:
product_id + "#" + YYYYMMDD

# Example:
```

```
prod_00000#20260925
```

Reverse time was calculated as 9,999,999,999 minus the Unix timestamp at session start. The user ID prefix allows user-specific row ranges; reverse time orders recent sessions earlier within the same user ID prefix, assuming consistent formatting.

4.3 How views and cart quantities were calculated

For each session, the loader checked page_views. A page with page_type == product_detail and a valid product_id increased views by one for (product_id, session_date). It also iterated over the final cart_contents and added item.quantity for each product-date pair. This final-cart metric is not necessarily the count of individual add-to-cart actions.

```
# Simplified explanation of the actual loader logic.
if page.get("page_type") == "product_detail" and product_id:
    metrics[(product_id, session_date)]["views"] += 1


for product_id, item in session["cart_contents"].items():
    metrics[(product_id, session_date)]["cart_quantity"] += int(item["quantity"])
```

Confirmed HBase load: 5,000 sessions and 16,804 product-date metric rows. These 5,000 sessions are the first records of sessions_0.json, a deterministic subset. Do not describe them as a random or statistically representative sample.

4.4 Shell command and actual observed example

```
"scan 'fp_product_metrics', {LIMIT => 3}" | docker exec -i hbase-recovery /hbase/bin/hbase shell -n
```

Row key	views	cart_quantity
prod_00000#20260830	1	0
prod_00000#20260925	1	3
prod_00000#20261019	1	2

The three HBase rows above came from the verified scan. The row key represents product + day; it is not a transaction identifier. The Docker container used during the work was hbase-recovery (image harisekhon/hbase).

4.5 Exports and parsing

HBase shell output was exported to integration/hbase_product_metrics.txt, showing 16,804 row(s). A Python regex parser extracted product_id, date, m:views and m:cart_quantity into integration/hbase_product_metrics.csv. All 16,804 rows were converted, with zero incomplete rows.

```
# Core parsing idea (illustrative regex fragment).
pattern = r"(prod_\d+)#(\d{8}).*column=m:(views|cart_quantity).*value=(\d+)"
# Convert YYYYMMDD into YYYY-MM-DD before writing CSV.
```

Icyitonderwa / Important: The HBase schema file comment mentions purchases, but the confirmed loader only wrote views and cart_quantity to fp_product_metrics. Correct that misleading comment before final submission.

5. Apache Spark setup and batch cleaning

Mu Kinyarwanda: Spark yadufashije gusoma records 500,000, kugenzura niba hari ibibura cyangwa transaction IDs zisubiramo, guhindura data types no kubika cleaned data muri Parquet. Kuba hatarabonetse duplicates cyangwa invalid rows ntabwo bivuze ko cleaning yari inutile; twakoze igenzura ry’ubuziranenge n’ihinduramatwara rya types.

5.1 What is Spark?

Apache Spark is a distributed data-processing framework. In the local project it ran with master("local[2]"), meaning it used two local worker threads. This demonstrates Spark DataFrames and SQL, but do not claim a production multi-node Spark cluster.

Environment seen in this workflow: Python 3.11.9; pyspark 4.2.1 installed, while C:\spark contained Spark 3.5.9. Keeping an older SPARK_HOME pointed at C:\spark caused a JavaPackage-related error earlier. Clearing SPARK_HOME in the PowerShell session resolved that version mismatch.

```
# Use in a new PowerShell session before launching PySpark scripts.
Remove-Item Env:SPARK_HOME -ErrorAction SilentlyContinue
python .\spark\spark_batch.py
```

5.2 Why NDJSON?

The original transactions.json was one large JSON array of 500,000 objects. prepare_transactions.py converted it to 500,000 newline-delimited JSON records in spark/data/transactions.ndjson (about 203 MB). Spark can read line-delimited JSON directly without requiring multi-line processing of a single large array.

```
# Read and inspect a Spark DataFrame.
transactions = spark.read.json("spark/data/transactions.ndjson")
transactions.printSchema()
print("Total transactions:", transactions.count())
```

5.3 Cleaning script explained

spark_clean.py checked selected fields for nulls/blanks and invalid totals, normalized status/payment_method, cast monetary columns from strings to Decimal(18,2), parsed transaction_time, removed duplicate transaction IDs and saved cleaned Parquet.

```
# Key transformations from the script.
from pyspark.sql import functions as F
from pyspark.sql.types import DecimalType

money_type = DecimalType(18, 2)
df = df.withColumn("total", F.col("total").cast(money_type))
df = df.withColumn("status", F.lower(F.trim(F.col("status"))))
df = df.withColumn(
   "transaction_time",
   F.to_timestamp(F.col("timestamp"), "yyyy-MM-dd'T'HH:mm:ss")
)
clean_df = clean_df.dropDuplicates(["transaction_id"])
clean_df.write.mode("overwrite").parquet("spark/data/transactions_clean.parquet")
```

The code fragment shows the main operations; it omits the full validity predicate for readability. Use the original spark_clean.py for the precise complete logic.

Quality check	Actual output
Original rows	500,000
Invalid rows under tested checks	0
Duplicate transaction IDs removed	0
Clean rows written	500,000

The validation checked required transaction ID, user ID, parsed time and nonnegative total; it did not exhaustively validate every nested item field. Financial types inside items remained strings in Spark's original inferred schema and would need explicit casting for item-level calculations.

5.4 Parquet vs CSV

Parquet is columnar and stores types, useful for efficient Spark processing. CSV is easy to exchange with other tools but types are read as text unless a schema or casting is provided. In this project, cleaned transactions were written as Parquet; cross-source intermediate exports used CSV.

6. Spark SQL and business results

Mu Kinyarwanda: Mu SQL twabaze transactions hashingiwe kuri status, tukareba uko ibikorwa byagabanyijwe mu mezi, ndetse tunahuza users na transactions hakoreshejwe user_id. COUNT(*) ibara records zose; COUNT(DISTINCT user_id) ibara abakiriya batandukanye. Iri tandukaniro rishobora kubazwa muri viva.

6.1 What is a temporary SQL view?

```
transactions_df = spark.read.parquet("spark/data/transactions_clean.parquet")
transactions_df.createOrReplaceTempView("transactions")

status_summary = spark.sql("""
    SELECT status, COUNT(*) AS total_transactions,
        ROUND(SUM(total), 2) AS total_transaction_value
    FROM transactions
    GROUP BY status
    ORDER BY total_transactions DESC
""")
```

createOrReplaceTempView registers a DataFrame under a SQL name within that Spark session. It does not automatically create a permanent MongoDB or HBase table.

6.2 Verified status summary

Status	Transactions	Transaction value
completed	317,055	264,428,273.18
delivered	61,226	67,830,723.08
processing	60,933	67,191,918.13
shipped	60,786	67,125,249.06
TOTAL	500,000	466,576,163.45

Interpretation: completed is the largest category by count and amount. Total transaction value covers all statuses; it is not independently verified cash received or net profit. No currency unit was established by the data provided; avoid inventing one.

6.3 Verified monthly transaction trends

Month	Transactions	Unique users	Transaction value
2026-08	169,268	10,000	157,886,330.43
2026-09	166,157	10,000	154,991,898.17
2026-10	164,575	10,000	153,697,934.85

Transaction volume falls across the three observed months. Because the synthetic dataset shows every user active each month, do not infer realistic customer loyalty or natural seasonal trends from this pattern alone.

6.4 How the users JOIN worked

```
SELECT
    SUBSTRING(u.registration_date, 1, 7) AS registration_month,
    COUNT(DISTINCT t.user_id) AS unique_customers,
    COUNT(*) AS total_transactions,
    ROUND(SUM(t.total), 2) AS total_transaction_value
FROM transactions t
INNER JOIN users u ON t.user_id = u.user_id
GROUP BY SUBSTRING(u.registration_date, 1, 7)
ORDER BY registration_month
```

JOIN links transaction facts with customer attributes using user_id. Verified groups had 1,510, 1,743, 1,653, 1,688, 1,657, 1,713 and 36 unique customers for February–August respectively: total 10,000. Their transaction counts sum to 500,000.

6.5 Distinct vs non-distinct count

COUNT(*) measures records. COUNT(DISTINCT user_id) counts different customers. A customer who makes 50 transactions contributes 50 to COUNT(*) but only one to COUNT(DISTINCT user_id). This distinction is fundamental to interpreting business data.

7. Cohort analysis and careful interpretation

Mu Kinyarwanda: Cohort ni itsinda ry'abakiriya bafite ikintu bahuriyeho, urugero ukwezi biyandikishirijemo. Twabonye 100% y'abakiriya bari active mu mezi atatu twasesenguye, ariko ibi bishingiye ku makuru yakozwe na generator. Ntukabyite retention nyayo yo mu bucuruzi cyangwa ngo wemeze ko abakiriya bose bazagaruka buri gihe.

7.1 What is a cohort?

A cohort is a group of users sharing a starting characteristic. In this project, cohort_month = registration_date[:7]. A customer registered in March 2026 belongs to the March cohort, regardless of when they made later transactions.

7.2 Formula

Active rate (%) = (distinct cohort users who have ≥1 transaction in a given activity month ÷ all users registered in that cohort) × 100.

```
# SQL expression used in the analysis:
ROUND(100.0 * a.active_customers / s.cohort_size, 2)
    AS active_rate_pct
```

7.3 Confirmed cohort sizes and active results

Registration cohort	Cohort size	Aug 2026	Sep 2026	Oct 2026
Feb 2026	1,510	100%	100%	100%
Mar 2026	1,743	100%	100%	100%
Apr 2026	1,653	100%	100%	100%
May 2026	1,688	100%	100%	100%

Jun 2026	1,657	100%	100%	100%
Jul 2026	1,713	100%	100%	100%
Aug 2026	36	100%	100%	100%

The result is 100% in every available activity month because all users in each cohort appear in at least one transaction in August, September and October. The results were saved as CSV under spark/output/cohort_analysis/. This metric is monthly transaction activity relative to registration cohorts, NOT a conventional month-0-to-month-N retention rate from registration, because transaction activity is only available for August–October in this dataset.

7.4 Common viva trap

Does 100% activity imply 100% real-world retention?

No. The dataset is synthetic, transactions occur only in a three-month observed window, and all transaction statuses counted as activity. The metric supports the statements about this generated dataset, not real-world loyalty.

Why are there only 36 users in August registration cohort?

That is what the generated registration dates produced in the data. The transcript does not establish its precise cause. Do not invent an explanation.

8. Integrated analytics: MongoDB + HBase + Spark

Mu Kinyarwanda: Aha ni ho datasets zacu zahuriye. Twafashe product catalog na transaction items muri MongoDB, dufata views na cart quantity muri HBase, duhuriza byose muri Spark dukoresheje product_id. Kubera LEFT JOIN, products 5,000 zose zaragumye, harimo na zimwe zitaragaragaye muri transaction items.

8.1 Business question and input files

Question: Which products attract browsing interest and how does this compare with transaction performance? MongoDB exported product catalog and product-level transaction summaries; HBase exported daily view and final-cart quantities; Spark aggregated HBase rows by product and joined all inputs using product_id.

Input	Source	Confirmed rows/products
mongodb_products.csv	MongoDB products	5,000
mongodb_product_sales.csv	MongoDB transaction items	4,766 products
hbase_product_metrics.csv	HBase product-date records	16,804
Integrated output	Spark left joins	5,000 products

8.2 Key JOIN steps

```
# Aggregate daily HBase rows into one row per product.
browsing_summary = browsing.groupBy("product_id").agg(
    F.sum("views").alias("total_views"),
    F.sum("cart_quantity").alias("total_cart_quantity")
)

# Keep every catalog product.
integrated = (
    products
    .join(browsing_summary, "product_id", "left")
```

```
    .join(sales, "product_id", "left")
)
```

The left joins start from the full products table. That preserves catalog products even if HBase has no record for them or if no transaction item mentions them. The project filled missing activity counts with zero to support comparison. Zero after a join means no matching observed record in the source coverage, not proof of never being viewed or purchased in the real world.

8.3 Verified top products

Metric	Product	Verified values
Highest sample browsing views	prod_01411	13 views; cart quantity 12; completed quantity 276
Highest completed item quantity	prod_01951	403 completed units; 9 sample views; cart quantity 11

The highest-viewed product in the sample is not the highest-completed-quantity product in the complete transaction export. This is a useful observation but is NOT evidence that views cause or fail to cause sales. The sources have very different denominators (5,000 sessions versus 500,000 transactions).

8.4 Reproducible key commands

```
python .\integration\convert_hbase_to_csv.py
python .\integration\export_mongodb.py
Remove-Item Env:SPARK_HOME -ErrorAction SilentlyContinue
python .\integration\spark_integrated_analysis.py
```

Confirmed integrated result: products 5,000; sales products 4,766; HBase product-date rows 16,804; integrated products 5,000; integrated CSV saved to integration/output/product_analysis/.

9. Visualizations and findings

Mu Kinyarwanda: Charts enye zidufasha gusobanurira mwarimu imibare tudakoresheje tables ndende. Iya mbere ireba views, iya kabiri cart quantity, iya gatatu completed quantity, iya kane igereranya views na completed quantities. Ibyo tubona si gihamya y'uko views zitera purchases, kuko sources zitapima abantu cyangwa igihe kimwe mu buryo bungana.

9.1 Four generated figures

Figure	Filename	Interpretation
1	chart1_top_views.png	Top 10 product IDs by HBase sample product-detail views
2	chart2_top_cart.png	Top 10 product IDs by HBase sample final-cart quantities
3	chart3_completed_quantity.png	Top 10 products by completed transaction item quantity from MongoDB
4	chart4_views_vs_completed.png	Scatter plot of sample views against completed quantity

All four PNGs were confirmed present in visualizations/charts. The fourth chart was visually reviewed: titles, axes and points were legible. Vertical strips of points are expected because the x-axis is an integer view count (e.g., 1, 2, 3, ...).

9.2 What to say while presenting each figure

1. Chart 1: “This chart ranks products by product-detail page views in the 5,000-session HBase subset; prod_01411 had 13 views.”
2. Chart 2: “This chart shows the quantities present in final shopping carts for the HBase subset. It is not completed purchases.”
3. Chart 3: “This chart ranks quantities contained in completed-status transactions; prod_01951 had 403 units.”
4. Chart 4: “We compare browsing views and completed quantities by product ID. Different data coverage means this is descriptive, not a conversion rate or causal test.”

9.3 Analytic cautions to memorize

- Never confuse total transaction value with net revenue, profit or cash collected.
- Never confuse final-cart quantity with completed transaction quantity.
- Never claim the first 5,000 sessions are a random sample.
- Never claim monthly active rate equals real-world retention.
- Never infer causal relationships from a scatter plot without a suitable design and statistical evidence.
- Do not report “overall conversion rate 20.29%” from an earlier draft unless a separate verified query and denominator establish it.

10. Implementation problems and troubleshooting

Mu Kinyarwanda: Muri project twagize ibibazo by’itangira rya HBase, SPARK_HOME itajyanaga na version ya PySpark, hamwe n’ubunini bwa data. Mu viva, ujye uvuga ikibazo, uko wagitahuye, uko wagikemuye n’isomo wakuyemo. Ibyo byerekana ko wumva system, aho kuba umuntu wakopeye code gusa.

Challenge observed	What happened	Resolution / lesson
MongoDB setup	MongoDB needed local configuration	Used mongodb://localhost:27017 and verified database/collection counts
HBase startup	Original container thrift startup was problematic	Recovered HBase with container hbase-recovery and confirmed shell queries
Version mismatch	SPARK_HOME pointed to Spark 3.5.9 while pyspark 4.2.1 was installed	Cleared SPARK_HOME in PowerShell before using installed PySpark
Large JSON files	500k transactions; 2M sessions	Used streaming ijson for conversion and batch parsing
Source coverage mismatch	HBase has 5k selected sessions vs full MongoDB transactions	Qualified integrated comparisons and avoided conversion claims
Synthetic cohort	All observed cohorts had 100% monthly activity	Reported exact result with its limitations

10.1 Normal warnings versus real failures

The warnings “Using incubator modules: jdk.incubator.vector”, native Hadoop fallback and the PySpark/pandas version FutureWarning were observed in successfully completed runs. A WARNING alone is not proof of failure. Look for stack traces, missing outputs, incorrect counts and nonzero process status.

10.2 Useful PowerShell checks

```
# Verify output artifacts.
Test-Path .\report\Technical_Report.docx
Test-Path .\report\Technical_Report.pdf
```

```
Test-Path .\report\Final_Project_Presentation.pptx
Get-ChildItem .\visualizations\charts\*.png

# Check HBase container.
docker ps --filter "name=hbase-recovery"

# Check Spark compatibility for this terminal.
Remove-Item Env:SPARK_HOME -ErrorAction SilentlyContinue
```

11. Reproducibility and submission checklist

Mu Kinyarwanda: Reproducibility bisobanura ko undi muntu yashobora gusubiramo ibikorwa byawe ahereye ku mabwiriza. README igomba kwerekana software, commands, structure ya files, uko databases zitegurwa n'ibisubizo bitegerejwe. Ntugashyiremo passwords cyangwa datasets nini mugihe bitasabwe. Banze wemeze ko GitHub na ZIP zihuye.

11.1 Run in a sensible order

5. Start in FinalProjectBD and verify Python 3.11, required libraries, disk space and data files.
6. Validate original generator and JSON outputs without unnecessarily regenerating 2.5 million records.
7. Start/verify MongoDB service and check ecommerce_db collections. Keep credentials out of README and ZIP.
8. Start/verify HBase container, tables and row counts. Use the documented HBase shell commands.
9. Prepare NDJSON and run Spark cleaning; then Spark SQL and cohort analysis.
10. Export product metrics from HBase and MongoDB; run Spark integrated analysis.
11. Generate charts; build and review architecture diagram, technical report, PDF and PowerPoint.
12. Create README with dependencies, commands, data assumptions, output locations and reproducibility caveats.
13. Create a submission ZIP matching the source files committed to GitHub; verify paths and avoid sensitive files.

11.2 Selected local commands

```
cd C:\Users\user\Desktop\FinalProjectBD
python .\spark\prepare_transactions.py
Remove-Item Env:SPARK_HOME -ErrorAction SilentlyContinue
python .\spark\spark_clean.py
python .\spark\spark_sql_queries.py
python .\spark\spark_cohort.py
python .\integration\convert_hbase_to_csv.py
python .\integration\export_mongodb.py
python .\integration\spark_integrated_analysis.py
python .\visualizations\make_charts.py
python .\report\make_architecture.py
python .\report\build_report.py
python .\report\build_presentation.py
```

Icyitonderwa / Important: These commands assume all scripts and inputs exist and that services are running. HBase table creation and data loading occur as separate prerequisites; rerunning schema creation on existing tables can fail. Check the script before overwriting outputs.

11.3 Lecturer checklist (based on the requirements discussed)

- MongoDB: data model, representative import, at least 2 substantive aggregation queries and verified outputs.
- HBase: sensible row keys/column families, sample loading, user-session query and evidence.
- Spark: cleaning, batch processing, complex SQL/joins, cohort or recommendation indicator.
- Integrated analytics: ONE clear business question combining sources with an explained workflow.
- At least 3–4 meaningful and explained visualizations.
- Technical PDF: 6–8 pages; architecture, method, results, tradeoffs, challenges, limitations.
- GitHub URL and matching ZIP: reproducible README/scripts; keep large raw data or local stores out if not requested.

11.4 Review the existing report before submission

The previously generated Technical_Report.docx contained an “overall conversion rate: 20.29%” statement. That value was NOT established by the console outputs supplied here. Verify it with a clear query and numerator/denominator, or remove it from the report and slides. Also ensure the PDF truly has 6–8 pages; a successful file conversion alone does not prove compliance.

A second issue: the current report builder may output many pages if it embeds four full-width charts. Adjust image widths, captions, spacing and page breaks while preserving content. Check document page count in Word or a PDF reader.

12. Explain the project in a viva

Mu Kinyarwanda: Mu gihe mwarimu akubajije, tangira uvuga ikibazo washakaga gukemura, data wakoresheje, impamvu wahisemo MongoDB/HBase/Spark, uko wazihuje, ibisubizo by’ingenzi n’imbogamizi. Niba akubajije impamvu HBase ifite sessions 5,000 gusa, sobanura ko ari deterministic subset atari random sample.

12.1 A 90-second presentation script (simple English)

“Our project is a distributed multi-model analytics system for e-commerce data. We generated a synthetic dataset containing 10,000 users, 5,000 products, 500,000 transactions and 2 million browsing sessions. We stored the full dataset in MongoDB. We used HBase for 5,000 browsing sessions and created 16,804 daily product metrics. Apache Spark processed and cleaned 500,000 transactions, then we performed SQL queries and cohort analysis. For integration, we exported product information and transaction quantities from MongoDB, exported browsing metrics from HBase and joined them in Spark by product ID. We successfully integrated 5,000 products and built four charts. Our main limitation is that HBase covers only a deterministic subset, so we describe patterns rather than claim a true conversion rate. This project taught us data modeling, scalable processing, integration and troubleshooting.”

12.2 Core viva questions and short answers

1. Why use MongoDB?

It stores flexible nested documents such as transactions with arrays of items and supports aggregation pipelines.

2. Why use HBase?

It models high-volume, row-key-oriented session activity using column families and qualifiers.

3. Why Spark?

Spark processes large datasets with DataFrames, SQL, grouping and joins; in this project it ran locally.

4. Is Spark a database?

Primarily no. It is a data processing engine. We read JSON/Parquet and wrote CSV/Parquet artifacts.

5. What is a row key?

A primary lookup/order identifier for an HBase row; ours used user + reverse time + session or product + date.

6. What is a column family?

A collection of related HBase columns defined when creating a table; we used info, pv, cart and m.

7. Why reverse timestamp?

To put more recent session keys earlier within a user prefix, supporting recency-oriented scanning.

8. Why product_id?

It is the common key linking catalog products, HBase browsing metrics and MongoDB transaction items.

9. Explain INNER JOIN.

Only rows with matching join keys remain. We used it between users and transactions.

10. Explain LEFT JOIN.

All left-side products remain, even if no matching browsing or sales summary is found.

11. Why use DISTINCT?

To count unique customers, rather than counting each transaction as a different customer.

12. What is a DataFrame?

A structured tabular dataset used by Spark for transformations and actions.

13. What is lazy evaluation?

Spark commonly builds a transformation plan, then executes it when an action such as count/show/write runs.

14. What is batch processing?

Analyzing stored datasets in finite jobs, unlike continuously processing live events.

15. Why NDJSON?

One JSON object per line is easier for Spark to scan than a single giant JSON array.

16. Why Parquet?

It is columnar and type-aware, useful for efficient downstream Spark reads.

17. Why Decimal instead of float?

Decimal represents currency-like values with controlled precision, avoiding common floating-point rounding issues.

18. What did cleaning remove?

Zero records, because tested invalid conditions and duplicate transaction IDs were not found.

19. Is data cleaning still meaningful if zero removed?

Yes. We validated required fields and standardized types and text, then documented the outcome.

20. What is a cohort?

Users grouped by a common starting period, here registration month.

21. Why is every observed cohort 100% active?

Every cohort user has at least one transaction in each observed month of the synthetic dataset.

22. Is that real retention?

No. It is monthly activity relative to registration cohorts over a limited observed period.

23. How many MongoDB documents?

2,515,025: 10,000 users + 5,000 products + 25 categories + 500,000 transactions + 2,000,000 sessions.

24. How many HBase product-date records?

16,804 from the 5,000 selected sessions.

25. How many integrated products?

5,000 after left joining catalog, browsing summaries and transaction summaries.

26. Why are 4,766 products in transaction items?

That is the count of distinct product IDs found when scanning all 500,000 transactions; 234 catalog products had no observed transaction item.

27. What is the top browsing product?

prod_01411: 13 views and cart quantity 12 in the sampled HBase metrics.

28. What is the top completed quantity product?

prod_01951: 403 units in completed-status transactions.

29. Can we compute a reliable conversion rate from current integrated views and quantities?

No: browsing uses 5,000 selected sessions and transaction quantity uses all 500,000 transactions; definitions and denominators do not align.

30. What was the hardest infrastructure issue?

HBase container startup required recovery, and PySpark had a SPARK_HOME version mismatch.

31. What makes a pipeline reproducible?

Documented inputs, versions, commands, dependencies, schemas, assumptions and known limitations.

32. Why don't we claim correlation from the chart?

The chart is descriptive with unequal source coverage. Formal correlation would need comparable observations and statistical computation.

33. What does $unwind do in MongoDB?

It transforms each array element, such as a transaction item, into a separate pipeline document.

34. What does groupBy do in Spark?

It forms groups by a key, such as product_id, before aggregate computations.

35. What would you improve next?

Load a well-designed representative session sample or all sessions, align time windows, verify user/session transaction links and add robust tests.

13. Practice quiz and hands-on revision

Mu Kinyarwanda: Gerageza gusubiza ibi bibazo utareba ibisubizo. Niba utabasha gusobanura JOIN, ROW KEY, GROUP BY, COHORT cyangwa itandukaniro rya views na purchases mu magambo yawe, subira mu bice bibanza. Nyuma ugerageze guhindura query imwe no gusobanura impinduka zayo.

13.1 Quiz: answer before looking at solutions

14. Q1. Name the five MongoDB collections and their document counts.
15. Q2. Why is fp_product_metrics keyed by product_id#YYYYMMDD?
16. Q3. What is the difference between item_value_all and sum(total)?
17. Q4. Why does the left join output 5,000 products when the sales export has 4,766?
18. Q5. What do 13 views and 276 completed units for prod_01411 actually refer to?
19. Q6. What does COUNT(DISTINCT user_id) measure?
20. Q7. Why is the 100% cohort activity result not a real retention estimate?
21. Q8. Give two reasons the chart4 scatter plot should not be interpreted causally.
22. Q9. Which commands help verify HBase Docker status and report existence?
23. Q10. What would you change to calculate meaningful funnel conversion?

13.2 Suggested answers

1. users 10,000; products 5,000; categories 25; transactions 500,000; sessions 2,000,000.

2. It groups all daily metrics for a product under a predictable row-key prefix.

3. Item subtotals are before transaction-level discounts; sum(total) aggregates transaction-level totals and may cover multiple items/statuses.

4. All 5,000 catalog records are preserved, with missing activity filled with zero.

5. 13 views are in 5,000 selected HBase sessions; 276 completed units are from the MongoDB transaction export.

6. The number of different users in a group, not the number of transactions.

7. The generated dataset has all cohorts active in the only three observed transaction months; it does not cover registration-month retention histories.

8. Unequal data coverage and no experiment/causal identification; also x counts are sparse discrete observations.

9. docker ps --filter "name=hbase-recovery" and Test-Path .\report\Technical_Report.pdf.

10. Use session-aligned browsing and completed outcomes for the same population/time window, define numerator/denominator and quality-check joins.

13.3 Small exercises you can do on your own

24. Write a MongoDB aggregation to count transactions by payment_method. Explain each stage.
25. Modify Spark SQL to show average transaction value by status (SUM(total)/COUNT(*) or AVG(total)).
26. From HBase metrics, calculate the total view count for prod_00000 across all available dates.
27. Use Spark to show the number of products with zero observed HBase views after joining.

28. Document what changes if you use all transactions rather than status == completed in product quantity ranking.

14. Glossary and final factual summary

Mu Kinyarwanda: Iki gice ni icy'ingenzi cyo gusubiramo amagambo ya tekiniki n'imibare nyakuri. Wibuke ingano ya datasets, umubare w'ibikorwa bya Spark, 16,804 product-date rows za HBase, 5,000 products zahujwe n'ingaruka z'uko data ari synthetic. Ntuvuge imibare itagaragaye muri outputs.

Term	Simple explanation
Document database	Stores self-contained records, often with nested arrays/objects.
Collection	Group of MongoDB documents.
Aggregation pipeline	Ordered stages that filter, unwind, group and sort documents.
Row key	Identifier for an HBase row.
Column family	Logical grouping of HBase qualifiers.
DataFrame	Spark's structured, distributed dataset abstraction.
Schema	Fields and their data types.
JOIN	Combines datasets using matching key values.
Cohort	Users grouped by a shared starting period.
Parquet	Columnar data format useful for analytics.
NDJSON	Newline-delimited JSON: one JSON record per line.
Batch processing	Computation over a finite stored dataset.
Synthetic dataset	Artificially generated data for testing or demonstration.
Conversion rate	Well-defined percentage of a population completing a target event.
Retention	Share of a cohort returning/remaining active in subsequent defined periods.
Reproducibility	Ability to regenerate results from documented inputs and steps.

14.1 Figures to memorize before the viva

Fact	Verified value
Total synthetic MongoDB documents	2,515,025
Users / products / categories	10,000 / 5,000 / 25
Transactions / sessions	500,000 / 2,000,000
Selected HBase sessions	5,000
HBase product-date metrics	16,804
Spark cleaned transactions	500,000
Spark invalid / duplicate IDs removed	0 / 0
Completed transactions	317,055
All-status transaction value	466,576,163.45
Products found in transaction items	4,766
Integrated catalog products	5,000
Top HBase sample views	prod_01411 (13 views)
Top completed quantity	prod_01951 (403 units)
Generated charts	4
PowerPoint slides reported	13

14.2 Final reminders

Present accurate achievements, but explain limitations before being asked. The technical report should remain within 6–8 pages; this study guide is a separate offline learning document. Verify the report's unsupported conversion figure and include the real student/group identification before submission. Keep evidence screenshots, reproducible code, GitHub and ZIP consistent.

You should be able to answer four questions without opening your code: What question did we ask? Why did we choose MongoDB, HBase and Spark? How did we join the three sources? What exactly can and cannot be concluded from our outputs?

END OF GUIDE